

AiSOC Technical Guide

Install, operate and integrate — on a workstation and on a server

AiSOC project · v1.0 · Released 2026-10-03

MIT licensed · github.com/beenuar/AiSOC

Who this is for

You are installing AiSOC, or you have installed it and something is not behaving. The guide is linear — each section assumes the one before it — but the troubleshooting tables are written to be read on their own at the moment something breaks.

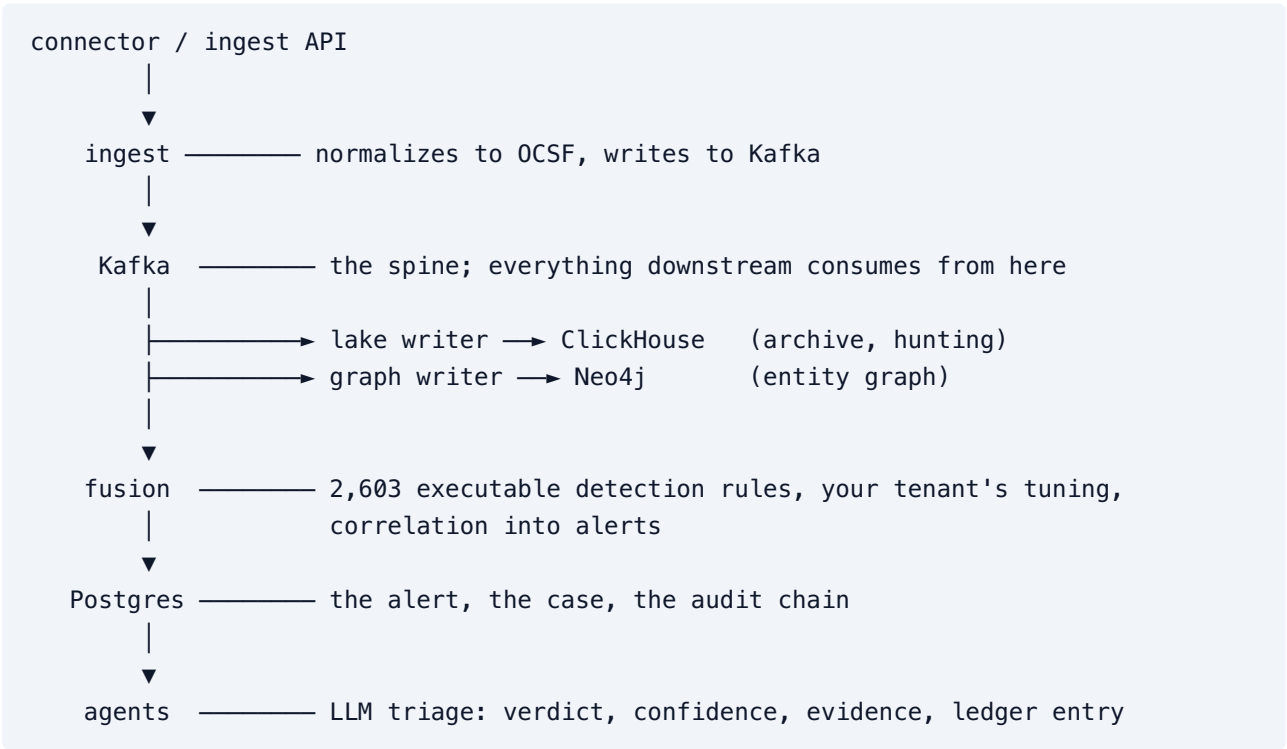
Three audiences, and the paths diverge early:

You are	Start at	Time
Evaluating on a laptop	Workstation install	~10 minutes
Deploying for a team	Server install	~45 minutes
Integrating with something	The REST API or MCP	—

1. What you are installing

AiSOC is a security operations platform: it ingests security telemetry, normalizes it, runs detections, correlates what fires into alerts, and has an AI agent triage them. It is not a SIEM replacement — it does not do long-term log retention or compliance search — and it does not discover telemetry you have not connected.

The pipeline, end to end



Deployment profiles

Profile	Services	Memory	Disk	What you get
CORE (make up)	16	8 GB	20 GB	The whole pipeline, AI triage with a local model, real threat intel. No credentials of any kind
full (make up-full)	22	12 GB	40 GB	CORE plus ClickHouse, Neo4j, Qdrant, UEBA — hunting, the entity graph, behavioural analytics

CORE is not a demo. It runs the real pipeline and a real local model; what it leaves out is the stores that only matter once you have volume.

Where the model runs

A model ships with AiSOC and runs on CPU, which is why `make up` needs no account, no key and no GPU. It is also the slowest of four options.

Option	Command	Works on
Bundled model, CPU	<code>make up</code>	everything
Bundled model, NVIDIA GPU	<code>make up-gpu</code>	Linux, Windows + WSL2
An Ollama you already run	<code>make up-host-llm</code>	everything, and the only GPU route on a Mac
A hosted provider	configured in the console	everything

`make doctor` tells you which fits the host you are on. The first-run wizard's **Choose where the AI runs** step tells you which is in effect right now.

The GPU reservation is an overlay, not a line in the base compose file, and that is deliberate rather than tidy. A host without an NVIDIA GPU and the container toolkit cannot start a service that reserves one — the daemon answers `could not select device driver "nvidia"` and refuses — so putting it in `docker-compose.yml` would take out first run for every Mac, every CPU-only Linux box and every CI runner.

`make up-gpu` runs `scripts/check_gpu_runtime.py` first, which exists because the daemon's own error names neither a cause nor a next command. The preflight distinguishes a missing card, driver, toolkit and daemon configuration, and names the install step for whichever is absent.

On Apple Silicon, `make up-gpu` cannot help and the preflight says so. Docker Desktop does not pass the Metal GPU into a Linux container — no toolkit, driver or setting changes that, so a container on a Mac is CPU-only whatever is reserved. A natively-installed Ollama does use Metal:

```
brew install ollama
OLLAMA_HOST=0.0.0.0 ollama serve          # in its own terminal
ollama pull llama3.2:3b-instruct-q4_K_M
make up-host-llm
```

OLLAMA_HOST=0.0.0.0 matters: Ollama binds loopback by default, which a container cannot reach.

`make up-host-llm` is worth running even without a GPU. `11434` is in the managed port inventory, so on a host already running Ollama, `make up` sees the conflict and republishes the bundled one on a free port — leaving you with two Ollamas, the stack talking to the new one, and yours idle.

Checking what is actually running

A GPU reservation is a **request**. A model can still land on the CPU for want of VRAM or a usable driver, so reading the compose file back would report the intent and call it the outcome. Ask instead:

```
curl -s localhost:8000/api/v1/llm/runtime -H "Authorization: Bearer $TOKEN"
docker compose exec ollama ollama ps      # SIZE and the GPU/CPU split
```

Five answers, and **not loaded right now** is one of them: Ollama unloads after a few minutes idle, so an empty answer means nobody has asked it anything yet. That is not the same as CPU, and reporting CPU there would be a guess about the exact thing you are deciding on.

placement	Means
gpu	the whole model is in VRAM
partial	split; <code>detail</code> says what fraction, which explains the latency
cpu	entirely on the CPU
unknown	nothing loaded, so it cannot say
unreachable	nothing answered — expected on a hosted-provider deployment

2. Workstation install

For evaluating, developing, or running a small deployment on a laptop.

Prerequisites

Requirement	Minimum	Why
Docker	24+ with Compose v2	The whole stack is containers
Memory allocated to Docker	8 GB for CORE, 12 GB for full	Below this, services restart-loop
Disk free in the Docker VM	20 GB	Kafka corrupts its log directory when it runs out, and its healthcheck still passes
<code>make</code> , <code>git</code> , <code>curl</code>	any	Driving the stack

On macOS the Docker VM has its own disk allocation, separate from your machine's. Docker Desktop → Settings → Resources is where you raise it, and "I have 500 GB free" is not the relevant number.

Install

```
git clone https://github.com/beenuar/AiSOC.git
cd AiSOC
make up
```

That is the whole thing. `make up` generates real secrets into `.env`, creates an administrator, and prints the password once.

Do not run `cp .env.example .env` first. It used to be a documented step and it made things worse: the example file carried an invalid Fernet placeholder, and the credential vault treats empty as "use an ephemeral dev key" while non-empty-and-invalid is fatal. Copying the file gave you HTTP 500 on every connector save, discovered at the connector wizard rather than at boot. Not copying it gave you a working vault.

Verify it came up

```
make doctor
```

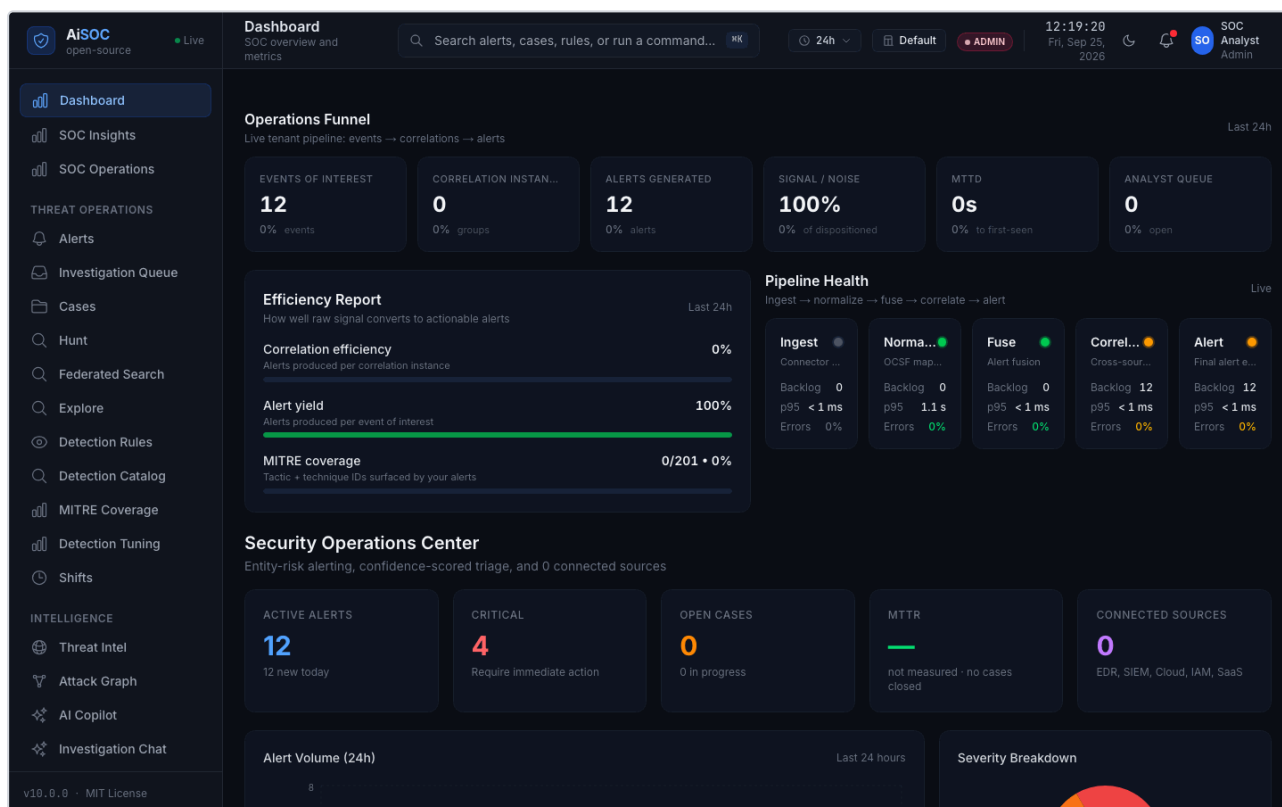
`doctor` queries each datastore rather than asking whether its container is up, which is the distinction that matters — a Kafka with a corrupt log directory reports healthy and moves nothing.

```
make smoke
```

One real event through the real spine: it mints an ingest token, confirms an unauthenticated write is refused, then watches the event be accepted, carried by Kafka, detected and promoted by fusion, and land as an alert in Postgres. If this passes, the product works on your machine.

First sign-in

Open `http://localhost:3000`. Sign in with the administrator `make up` created — the password was printed once during install, and `make bootstrap ARGS=--reset-password` mints a new one if you lost it.



The operations funnel over real counts on a fresh install. Note *MTTR* reads not measured • no cases closed rather than 0 — an average over zero rows is not zero, and publishing it as zero would claim instant resolution.

Workstation troubleshooting

Symptom	Cause	Fix
port is already allocated	Something else holds 5432, 6379, 9092 or 3000	<code>make doctor</code> names the process or container. Postgres on 5432 and Ollama on 11434 are the two most likely, and the installer reassigns rather than refusing
Kafka healthy, nothing flows	Docker VM out of disk. Kafka corrupts its log dir and the healthcheck still passes	<code>docker system prune -af</code> , then <code>make clean && make up</code>
<code>make smoke</code> fails at "became an alert"	fusion is down or not consuming	<code>docker compose logs fusion \</code> <code>tail -60</code>
Console loads but is empty	No data yet. This is correct	<code>make smoke</code> , or load sample data from the setup wizard
Services restart-loop	Not enough memory for <code>full</code>	Use <code>make up</code> (CORE) and raise Docker's allocation before retrying
<code>credential vault</code> unavailable saving a connector	<code>AISOC_CREDENTIAL_KEY</code> is set to something that is not a valid Fernet key	<code>make env</code> generates a real one, then <code>docker compose up -d api</code>
<code>make doctor</code> shows 22 red checks	You have not run <code>make up</code> yet	Run it. <code>doctor</code> says this explicitly rather than listing failures
<code>make doctor</code> warns that Ollama holds 11434	You already run Ollama	Not a problem. <code>make up-host-llm</code> uses yours instead of starting a second one; <code>make up</code> republishes the bundled one on a free port
<code>make up-gpu</code> refuses before compose starts	The NVIDIA container toolkit is missing, or this is a Mac	The preflight names which of the four pieces is absent. On Apple Silicon no install helps — use <code>make up-host-llm</code>
Triage is slow	The model is on CPU, which is the default	<code>GET /api/v1/llm/runtime</code> confirms it. <code>make up-gpu</code> , <code>make up-host-llm</code> , or a hosted provider
AI verdicts stopped after adding a provider key	The key is wrong, revoked, or names a model the account cannot use	Settings → Deployment & AI → Test. It distinguishes a bad key from a wrong model name, which are different fixes

Windows

`install.ps1` handles Windows 10 and 11 through `winget`. Run it from an elevated PowerShell. It creates the administrator and runs the smoke test, the same as the Unix path — an earlier version handed off to a demo stack and printed "AiSOC is up and running" without doing either.

3. Server install

For a deployment a team will use.

Sizing

Deployment	vCPU	RAM	Disk	Notes
CORE, evaluation	4	8 GB	50 GB	Fine for a few connectors
full, small team	8	16 GB	200 GB	ClickHouse grows with retention
full, production	16	32 GB	500 GB+	Size the lake from your event rate × retention

Event volume drives the lake more than anything else. A deployment taking 1,000 events/second at 1 KB each writes roughly 86 GB/day before compression.

Install

```
git clone https://github.com/beenuar/AiSOC.git
cd AiSOC
make up-full
```

Then, before anyone else touches it:

```
make doctor
make smoke
```

Make it production-class

`make up` is a development posture. Four changes make it a deployment:

1. Set the environment. The default is `development`, and `AUTH_BYPASS_ENVIRONMENTS` contains `development`, which means a request with no credentials is served as an administrator.

```
ENVIRONMENT=production
```

This is not theoretical. The default shipped that way, and because `bootstrap_admin`'s tenant was byte-identical to the demo tenant, an anonymous caller landed in the real administrator's tenant rather than a sandbox. Both defaults now read `production`, and the two tenant ids are asserted different at runtime — but a deployment that overrides it back is back in that state.

2. Put TLS in front of it. AiSOC serves plain HTTP and expects a reverse proxy to terminate TLS. Nothing in the stack does it for you.

3. Keep the secrets `make up` generated. They are in `.env`, they are real, and they are per-deployment. Back the file up somewhere that is not the server.

4. Set the CORS origin to your console's real address. The default is permissive enough for localhost and wrong for anything else.

```
AISOC_CORS_ORIGINS=https://soc.yourcompany.example
```

The shared CORS module **refuses to start** when `AISOC_CORS_ORIGINS` contains `*` while credentials are enabled and `AISOC_ENV=production`, which is the one misconfiguration here that cannot be made quietly.

Server troubleshooting

Symptom	Cause	Fix
Console loads, every API call 401s	<code>ENVIRONMENT=production</code> with no administrator created	<code>make bootstrap ARGS=--reset-password</code>
Connectors save but never poll	The scheduler registered every job paused	Fixed in current releases. Check <code>last_sync</code> on the connector: a null that never changes is this
Alerts arrive with source <code>unknown</code>	The connector has no ingest profile, so it fell through to the lenient fallback	Check <code>connectorProfiles</code> in <code>services/ingest/internal/normalizer/normalizer.go</code>
Events ingest but no alerts appear	The fallback classifies events as OCSF category 4, and promotion needs category 2 or <code>severity_id >= 4</code>	Same cause as above. A vendor finding that already passed the vendor's correlation belongs at <code>class_uid: 2001</code>
<code>/api/docs</code> returns 404	<code>make up</code> starts a production-class stack with the spec disabled	This is intended. Use <code>docs/openapi.yaml</code> in the repository
Everything is slow under load	One connection acquire per alert write	Current releases batch; check you are not pinned to an older image

Upgrading

```
git pull
make up          # migrations run automatically
```

Migrations are idempotent and verified across three consecutive full passes in CI. Before a major upgrade, take a backup and **verify it**:

```
./scripts/backup.sh --component all
./scripts/restore.sh --timestamp <ts> --component all --dry-run
```

The dry run fetches, verifies and decrypts every artefact without writing. A backup you have never restored is a hypothesis.

4. Connecting real data

Two ways in: push to the ingest API, or configure a connector that pulls.

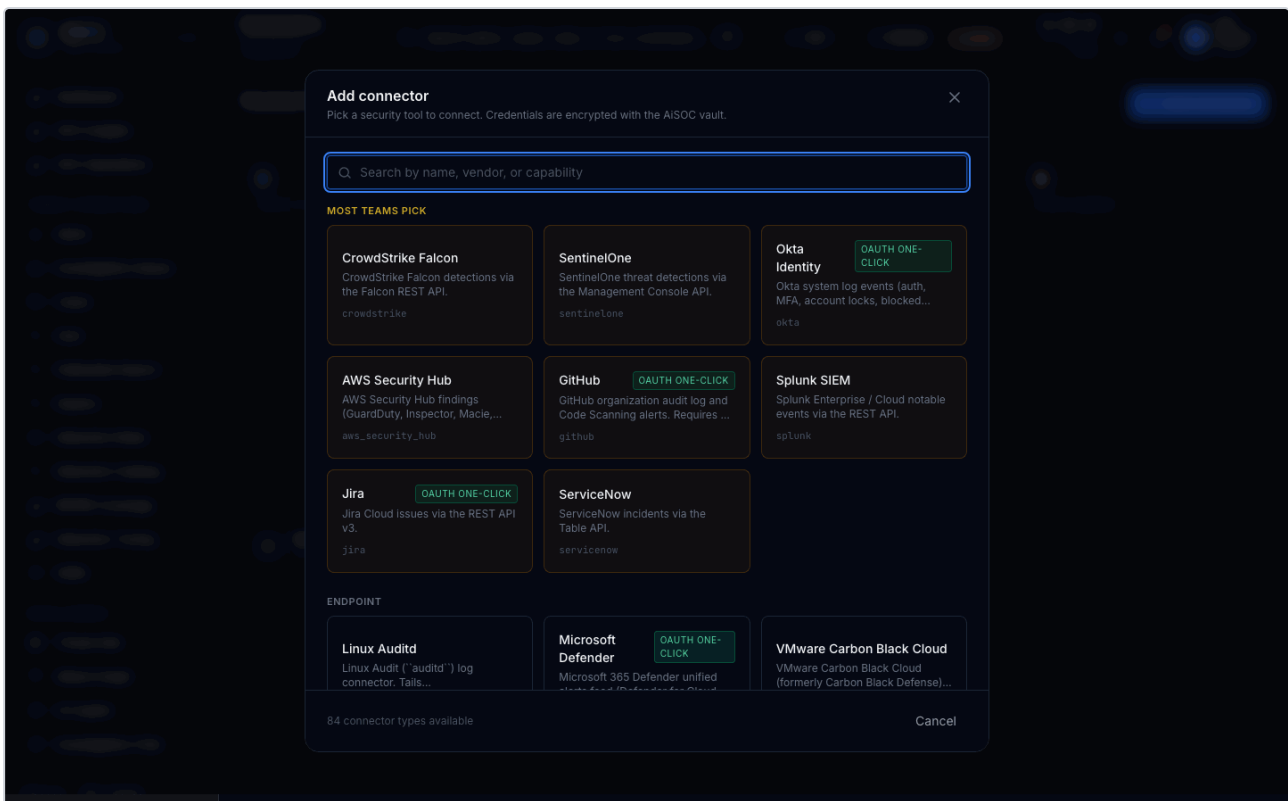
Push

```
export AISOC_INGEST_TOKEN=$(make ingest-token)

curl -X POST http://localhost:8081/v1/ingest/batch \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $AISOC_INGEST_TOKEN" \
  -d '{
    "connector_id": "edr-1",
    "connector_type": "crowdstrike",
    "events": [{
      "severity": "high",
      "title": "Encoded PowerShell from Office",
      "host": "WIN-FIN-01",
      "user": "a.hassan@example.com"
    }]
  }'
```

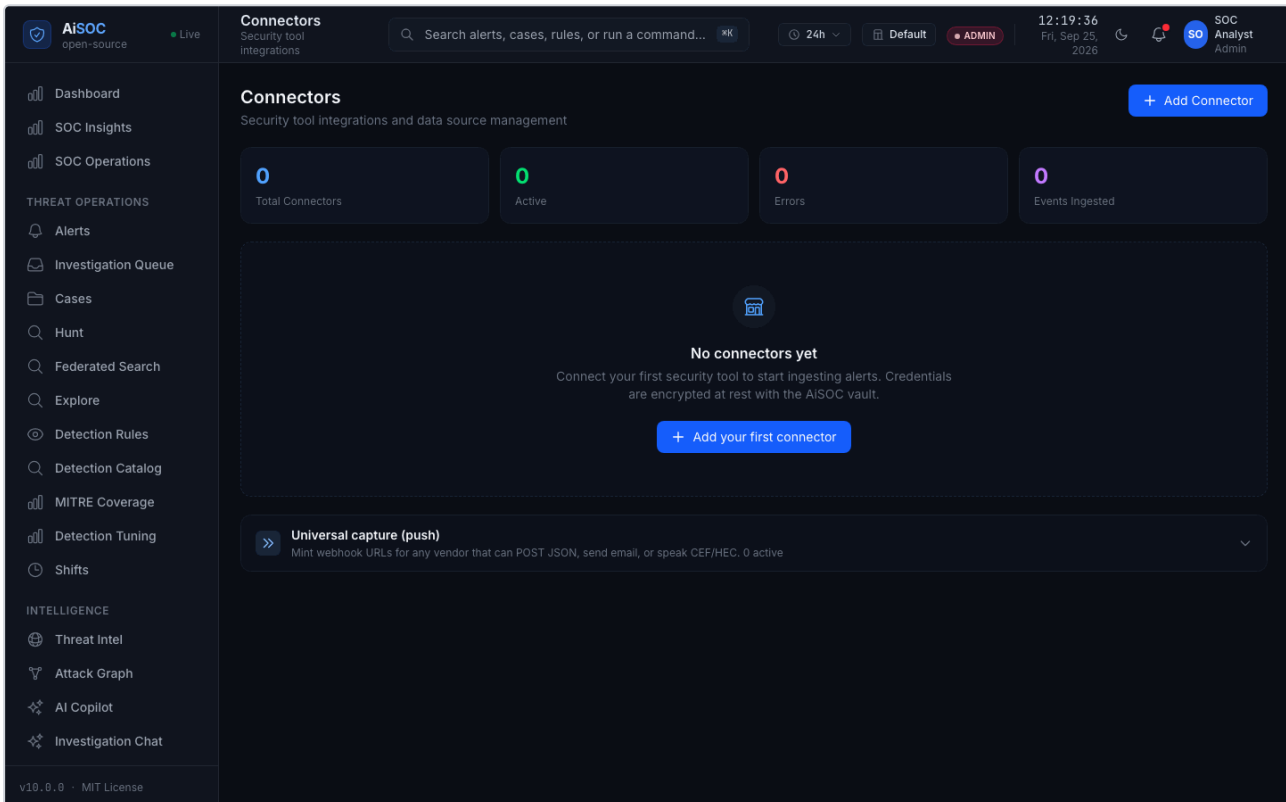
The tenant comes from the token, not from a header. It used to come from a caller-supplied `X-Tenant-ID` that was never checked against the tenants table, on an endpoint with no authentication at all.

Pull



The connector wizard, reporting 84 connector types available — generated from the registry, not hand-typed.

Settings → Connectors → Add connector. Credentials are encrypted at rest with Fernet before they touch the database.



An honest empty state. Nothing is invented to fill the page.

Connecting a source is meant to be the only manual step: the scheduler polls on a five-minute default, events flow to ingest, fusion promotes what matters, and the agent triages every fused alert. No further intervention.

Bringing existing detections

`packages/aisoc-migrate` translates Splunk SPL, Sentinel KQL and Elastic EQL:

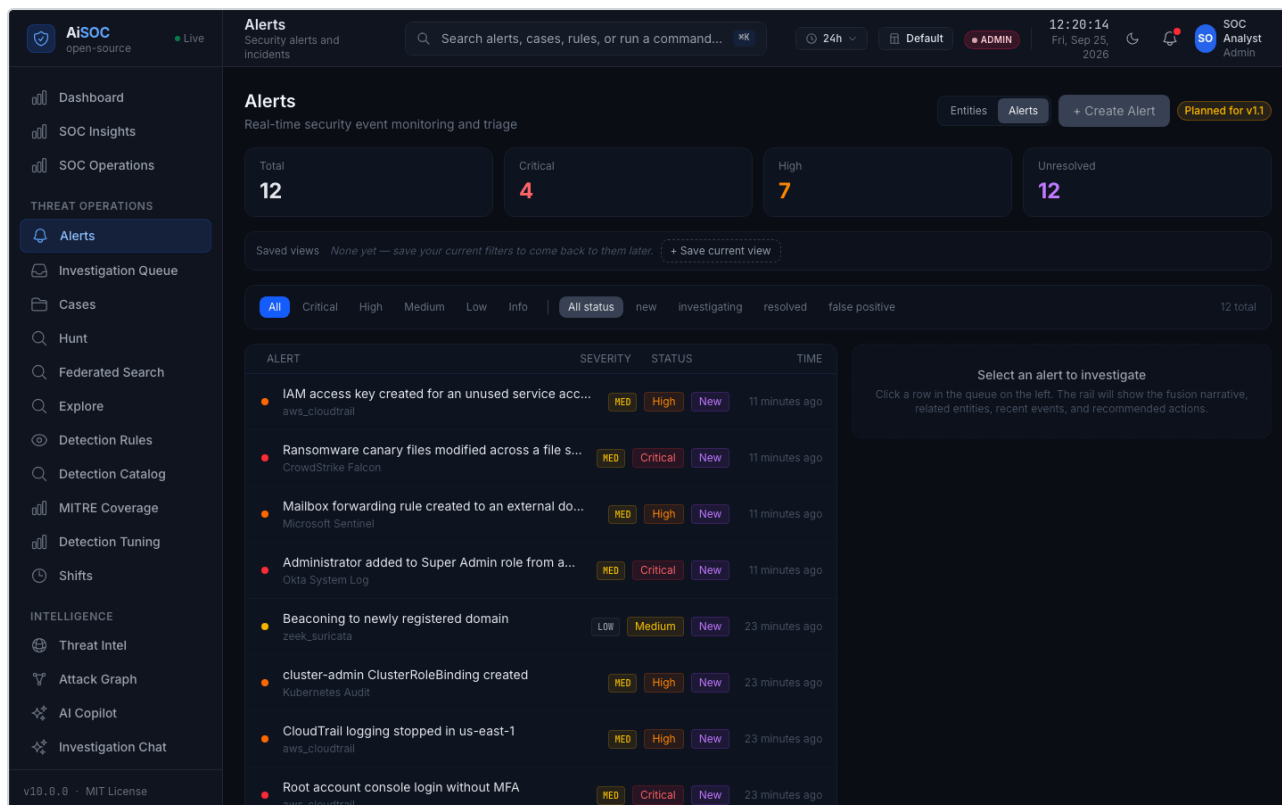
```
python3 packages/aisoc-migrate/inbound.py --dialect spl --file my-searches.txt
```

It **refuses rather than approximating** what it cannot carry — `transaction`, `lookup`, `join`, `sequence` and six others. An almost-right detection is harder to find than a missing one: it sits in the catalogue, fires on the wrong thing, and nobody checks it against the original.

Measured on the 2,005 quarantined Splunk rules bundled in this repository: 1,734 translate, and **1,711 of those are partial** — field matches carried, thresholds did not, because the corpus is aggregation-heavy. Read 86.5% as "a useful head start", not "a finished migration".

5. Using it

The alert queue



Twelve alerts the pipeline produced from pushed telemetry, each attributed to the connector that fed it.

Alerts are ordered by a priority score: severity multiplied by asset criticality, identity privilege and known-exploited-vulnerability exposure. Context multiplies and never sums, so it reorders alerts of similar severity — which is what it is for — and cannot push an informational alert above a critical one.

Every factor that moved the score is recorded with its contribution. An ordering you cannot interrogate is one you stop trusting the first time it surprises you.

The investigation rail

The screenshot displays the AiSOC investigation rail interface. The top navigation bar includes the AiSOC logo, a 'Live' status indicator, a search bar, a 24h time filter, a 'Default' view selector, a user profile for 'ADMIN', and the current time '12:20:19 Fri, Sep 23, 2026'. The left sidebar contains a navigation menu with sections for 'Dashboard', 'SOC Insights', 'SOC Operations', 'THREAT OPERATIONS' (highlighted), 'Investigation Queue', 'Cases', 'Hunt', 'Federated Search', 'Explore', 'Detection Rules', 'Detection Catalog', 'MITRE Coverage', 'Detection Tuning', 'Shifts', 'INTELLIGENCE', 'Threat Intel', 'Attack Graph', 'AI Copilot', and 'Investigation Chat'. The main content area is titled 'Alerts' and shows 'Real-time security event monitoring and triage'. It features a summary of 12 total alerts, with 4 Critical, 7 High, and 12 Unresolved. Below this is a 'Saved views' section and a filter bar for 'All status' (new, investigating, resolved, false positive). The alert list table has columns for ALERT, SEVERITY, STATUS, and TIME. The selected alert, 'Ransomware canary files modified across a file s...', is shown in detail on the right. This detail view includes a 'Story' tab, a 'Progression' section stating 'No ATT&CK tactic is mapped to this alert, so there is no progression to show. That is a gap in the detection's metadata rather than a statement about the activity.', an 'Evidence' table, and a 'Confidence' score of 61/100 (medium).

ALERT	SEVERITY	STATUS	TIME	
IAM access key created for an unused service acc... aws_cloudtrail	MED	High	New	11 minutes ago
Ransomware canary files modified across a file s... CrowdStrike Falcon	MED	Critical	New	11 minutes ago
Mailbox forwarding rule created to an external do... Microsoft Sentinel	MED	High	New	11 minutes ago
Administrator added to Super Admin role from a... Okta System Log	MED	Critical	New	11 minutes ago
Beaconing to newly registered domain zeek_suncata	LOW	Medium	New	23 minutes ago
cluster-admin ClusterRoleBinding created Kubernetes Audit	MED	High	New	23 minutes ago
CloudTrail logging stopped in us-east-1 aws_cloudtrail	MED	High	New	23 minutes ago
Root account console login without MFA aws_cloudtrail	MED	Critical	New	23 minutes ago

EVIDENCE	
Connector	1
CrowdStrike Falcon	1
Host	1
Rule	1
User	1

CONFIDENCE	
61/100	(medium)

The rail's Story view. Note the progression line: "no ATT&CK tactic is mapped ... that is a gap in the detection's metadata rather than a statement about the activity." The product distinguishes what it does not know from what is not there.

AI triage

The screenshot displays the AiSOC (AI Security Operations Center) interface. The top navigation bar includes the AiSOC logo, a 'Live' status indicator, a search bar, and user information for 'SOC Analyst Admin'. The left sidebar contains a menu with options like Dashboard, SOC Insights, SOC Operations, Alerts, Investigation Queue, Cases, Hunt, Federated Search, Explore, Detection Rules, Detection Catalog, MITRE Coverage, Detection Tuning, Shifts, Threat Intel, Attack Graph, AI Copilot, and Investigation Chat. The main content area is titled 'Alerts' and shows a summary of 12 total alerts, with 4 critical, 7 high, and 12 unresolved. Below this is a table of alerts with columns for Alert, Severity, Status, and Time. One alert is highlighted: 'Ransomware canary files modified across a file share' by CrowdStrike Falcon, with a severity of Critical and a status of New. To the right of this alert is a detailed view showing an automated triage verdict of 'true positive' with a confidence of 80/100, and a narrative explaining the alert's critical severity and the presence of IOCs.

A verdict from the bundled local `llama3.2:3b-instruct-q4_K_M`: true positive, confidence 80/100, groundedness not assessed, and the model's own rationale verbatim. No hosted provider was configured.

What the agent does and does not do:

- **It reads** the alert, its correlated siblings, entity context, and prior verdicts for the same signature.
- **It calls typed tools** — lake queries, graph traversals, enrichment lookups. The model picks the tool and passes arguments; it never writes SQL.
- **Everything is logged** to the Investigation Ledger: prompts, tool calls, citations, verdict, token cost. It exports as a signed bundle.
- **Grounding is checked on both paths.** A verdict citing an indicator the evidence never contained is demoted to human review rather than auto-closed. On the investigation path — the one you launch deliberately — nothing auto-closes, so the intervention is different: the reported confidence drops to the measured groundedness and a caveat naming the unsupported indicators is written into the summary you read.
- **No vendor is touched without a human**, unless a tenant has explicitly granted autonomy for that verb.

The gate **refuses to score an empty evidence set**, and reports `groundedness: null` rather than `0.0`. The distinction is the whole point: a case opened by hand carries no telemetry, every indicator the model mentions would read as unsupported, and a gate that scored anyway would

demote everything while appearing to catch hallucination. `null` means not measured; `0.0` means measured and nothing was supported.

Using your own provider

Seven providers are supported — `openai`, `anthropic`, `azure-openai`, `local-ollama`, `local-vllm`, `local-litellm`, `custom`. Configure one per tenant in **Settings → Deployment & AI**, or from the first-run wizard. The key is encrypted at rest with the credential vault and never returned by the API; reads report only `has_api_key`.

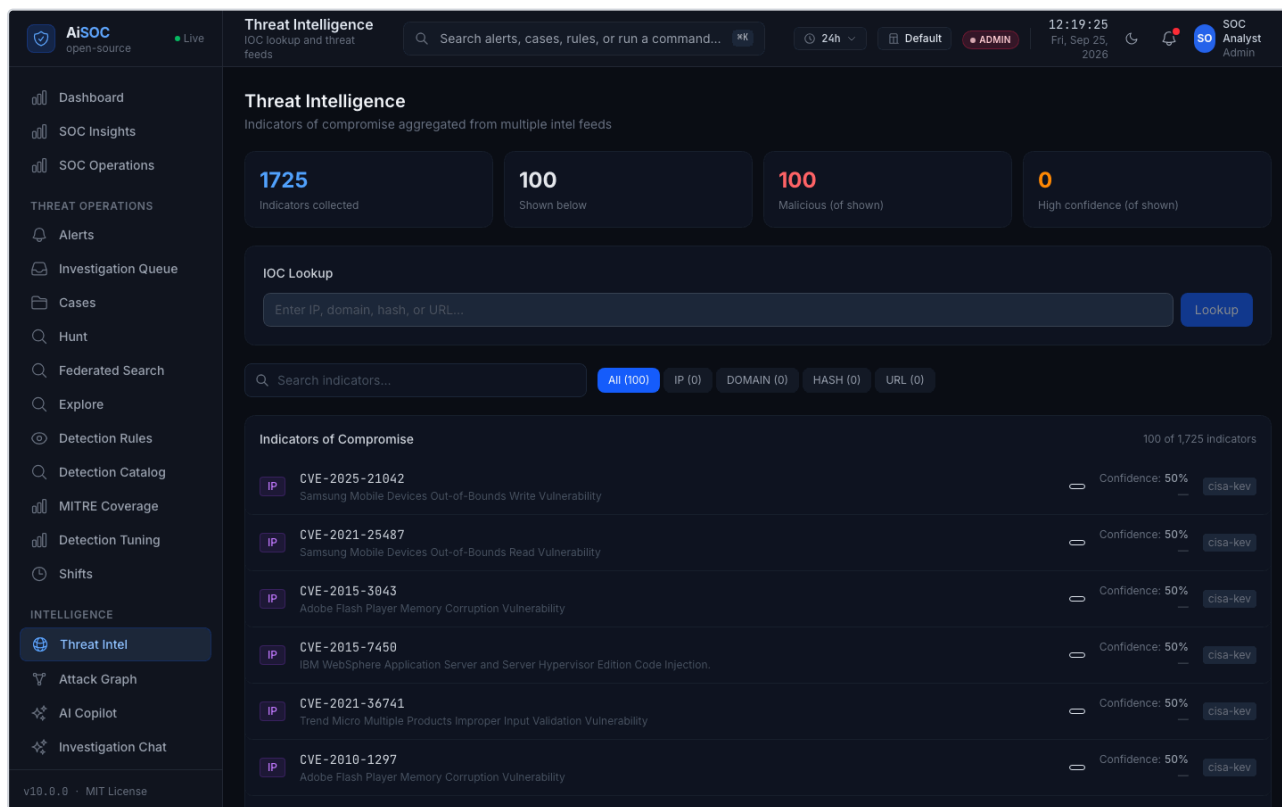
Test it before you rely on it. The Test button places one real one-token completion and reports what happened:

```
curl -sX POST localhost:8000/api/v1/llm/credentials/test \
-H "Authorization: Bearer $TOKEN"
```

Outcome	Means
<code>ok</code>	the provider answered. A <code>429</code> counts — being rate-limited means you reached it and were authenticated
<code>refused</code>	reached it and it said no. <code>detail</code> distinguishes a bad key from a wrong model name, which are different fixes
<code>unreachable</code>	nothing answered at that address
<code>unverified</code>	air-gapped, so the call was not attempted — the policy working, not a bad key

Before this existed, the credential routes validated only shape — that the URL parses, that the provider/key/base-URL combination is consistent — and none of them talked to the provider. A revoked key surfaced as triage quietly falling back to the deterministic path, which is a failure that is hard to attribute to a credential precisely because it is silent.

Threat intelligence



1,725 real Known Exploited Vulnerabilities entries, fetched from the public CISA catalogue. Not synthetic, not ours, unmodified — and the page distinguishes the catalogue size from the page it is showing.

Hunting

AISOc open-source • Live

Threat Hunting Proactive threat hunts and queries

Search alerts, cases, rules, or run a command... **24h** **Default** **ADMIN** 12:20:05 Fri, Sep 25, 2026 SOC Analyst Admin

Hunt at the speed of thought.

Ask in plain English and we'll translate it to ES|QL, KQL, and SPL. Save the questions that matter, schedule the ones that should run themselves.

Q e.g. Show me suspicious sudo from contractors in the last 4 hours **Translate & run →**

TRY ONE OF THESE

- Did we get any new attacks from Iran?
- Show me everyone who touched our prod IAM role in the last 7 days
- Any GitHub auth from a new device this week?

SAVED HUNTS + Ask

No saved hunts yet

Ask a question above and click Save to add it here.

SAVED SEARCHES + New

Couldn't load saved searches

We couldn't load this view. Try again or check the service status.

KQL Lucene SQL ES|QL Last 24 hours Save Run hunt

```
1 // Find suspicious PowerShell with encoded payloads
2 process where process.name == "powershell.exe"
3   and (process.command_line like~ "*-enc*"
4     or process.command_line like~ "*IEX*"
5     or process.command_line like~ "*DownloadString*")
6 | extend host=host.name, user=user.name
7 | project @timestamp, host, user, process.command_line
8 | sort @timestamp desc
```

Hunt results

Press Run to begin

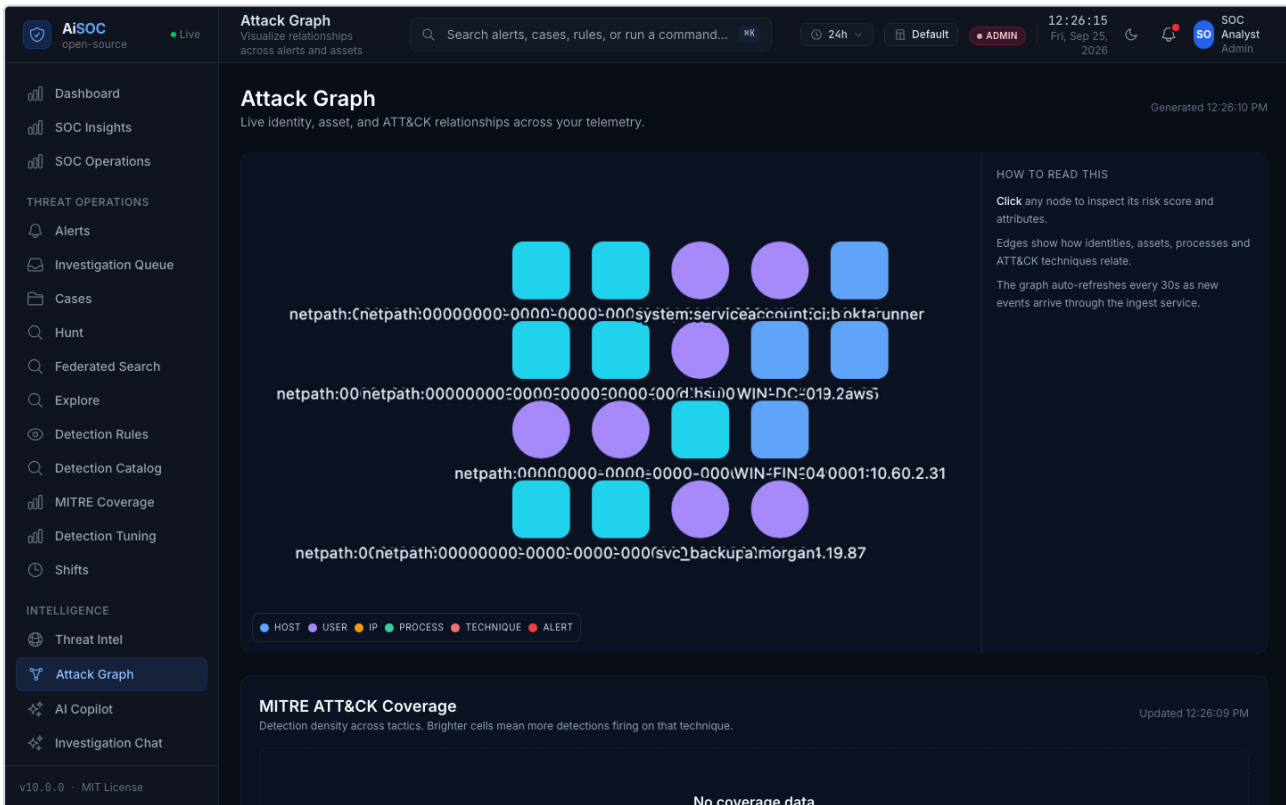
Tip: ask a question above, pick a saved hunt, or pivot from an alert.

v10.0.0 • MIT License

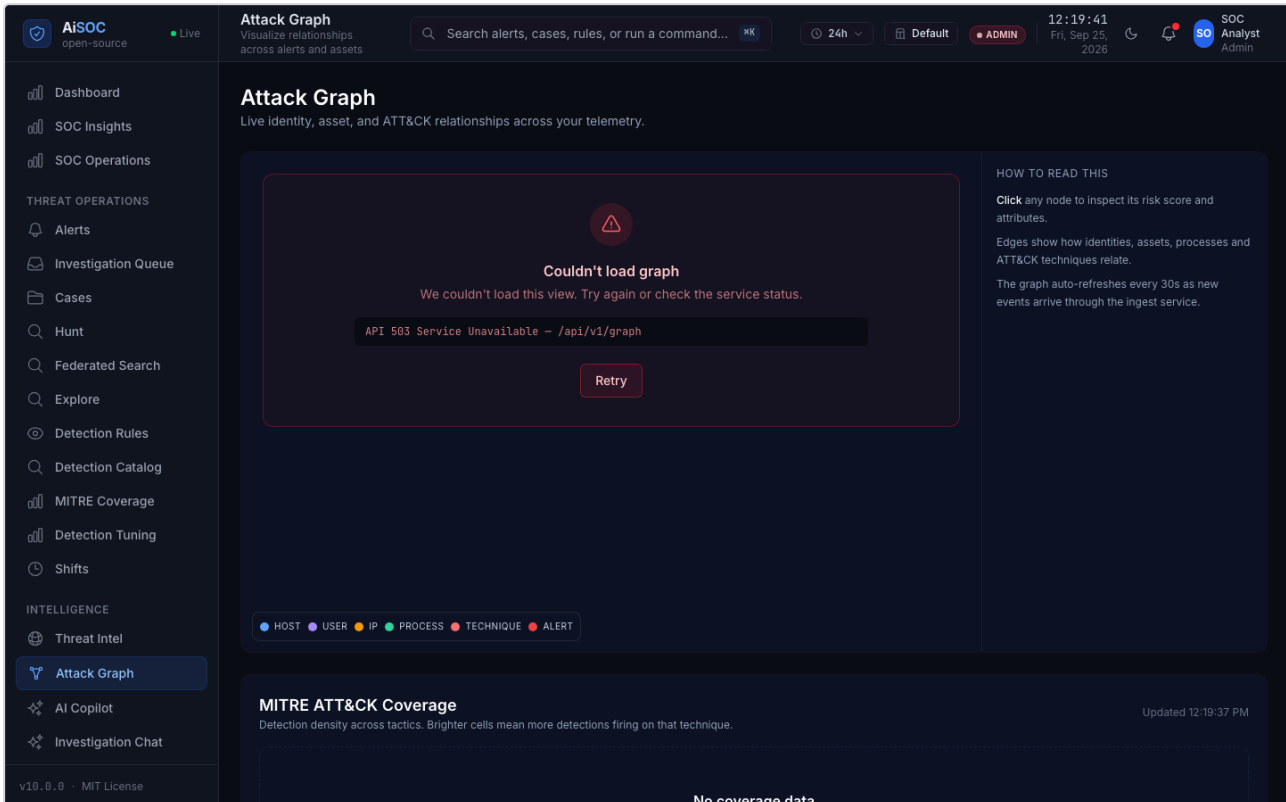
Natural language becomes a templated ES|QL, SPL or KQL query. The agent never writes raw query text.

Hunting needs the `full` profile, because it reads the ClickHouse lake.

The entity graph



The graph over real ingested telemetry, with Neo4j running.



The same page in CORE, where Neo4j is not running. It names the failure — `API 503 ... /api/v1/graph` — instead of drawing an invented graph. This is as much of the product as the populated view.

Playbooks

The screenshot shows the AISOCS Playbooks interface. The left sidebar contains navigation links for Dashboard, SOC Insights, SOC Operations, Threat Operations (Alerts, Investigation Queue, Cases, Hunt, Federated Search, Explore, Detection Rules, Detection Catalog, MITRE Coverage, Detection Tuning, Shifts), Intelligence (Threat Intel, Attack Graph, AI Copilot, Investigation Chat), and version information (v10.0.9 - MIT License). The main content area is titled 'Playbooks' with the subtitle 'Automated response workflows triggered by alerts and cases'. It includes a search bar, a '24h' filter, and a 'Default' button. A 'Draft from prompt' button and a '+ New Playbook' button are visible. Below the title, there are tabs for 'Playbooks (—)', 'Run History', and 'Community'. A message states 'Saved views: None yet — save your current filters to come back to them later.' with a '+ Save current view' button. A yellow error box displays the message: 'Playbooks unavailable: The agents service returned 503. The playbook list is unknown, not empty.' with a 'Retry' button. Below the error box, a larger box states: 'The playbook list could not be loaded. Treat this as unknown rather than as a tenant with no playbooks.'

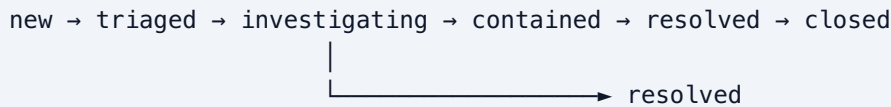
The screenshot shows a 'Choose Step Type' dialog box with a grid of step types. Each step type is represented by a colored box with a code and a description. The steps are: enr (Enrich), inv (Investigate), ntf (Notify), blk (Block IP), ioc (Block IOC), iso (Isolate host), tkt (Create ticket), cls (Close case), http (HTTP request), if (Condition), osq (Live query), dis (Disable user), pwd (Reset password), ses (Revoke sessions), mfa (Force MFA), kill (Kill process), qrn (Quarantine file), av (Run AV scan), scr (Run script), siem (Search SIEM), and note (Create notable event). A 'Cancel' button is at the bottom.

Code	Description
enr	Enrich
inv	Investigate
ntf	Notify
blk	Block IP
ioc	Block IOC
iso	Isolate host
tkt	Create ticket
cls	Close case
http	HTTP request
if	Condition
osq	Live query
dis	Disable user
pwd	Reset password
ses	Revoke sessions
mfa	Force MFA
kill	Kill process
qrn	Quarantine file
av	Run AV scan
scr	Run script
siem	Search SIEM
note	Create notable event

Twenty-one tiles from a twenty-two-member vocabulary. `approval` is withheld where the engine has no durable pause — a step that cannot suspend should not be offered.

The case lifecycle

Six states, and the machine is forward-only:



Two readings catch people out. **resolved is not terminal** — it means the case has an outcome, not that it has been closed out; only `closed` writes `closed_at`, which every duration metric reads. And **open is not a state**: it belonged to a pre-consolidation vocabulary and the database rejects it, so a filter naming it returns nothing rather than erroring, which is why it took a while to notice.

Forward-only is deliberate. It is what makes "this case was closed" mean something, and without it a title edit that happens to carry a status could walk a case backwards silently. So reopening is its own act, with its own record:

```
curl -sX POST "localhost:8000/api/v1/cases/$CASE_ID/reopen" \
  -H "Authorization: Bearer $TOKEN" -H 'content-type: application/json' \
  -d '{"reason": "The indicator reappeared on two more hosts overnight."}'
```

It requires a reason, increments `reopen_count`, and clears `closed_at` and `resolved_at` so a case cannot be counted as terminal and active at once. `reopen_count` exists because `reopened_at` is overwritten each time — a case reopened four times is a different conversation from one reopened once.

Honest empty states

The screenshot shows the 'Cases' page in the AiSOC interface. The left sidebar contains navigation items: Dashboard, SOC Insights, SOC Operations, THREAT OPERATIONS (Alerts, Investigation Queue), Cases (selected), Hunt, Federated Search, Explore, Detection Rules, Detection Catalog, MITRE Coverage, Detection Tuning, Shifts, INTELLIGENCE (Threat Intel, Attack Graph, AI Copilot, Investigation Chat). The main content area is titled 'Cases' with the subtitle 'Manage security investigations and incidents'. It features a search bar, a '24h' filter, and a 'Default' view. Below these are five status filters: All Cases (0), Open (0), In Progress (0), Resolved (0), and Closed (0). A 'Saved views' section shows 'None yet' with a '+ Save current view' button. A search bar for cases is present, along with a filter for 'All Severities' and a count of '0 cases'. The main content area displays a message: 'No cases yet. Cases bundle related alerts, evidence, and analyst notes into a single investigation. Promote alerts to a case from the Alerts tab, or click 'New Case' to start a manual investigation.' A '+ New Case' button and a 'Planned for v1.1' badge are visible in the top right of the main area. The footer shows 'v10.0.0 - MIT License'.

The screenshot shows the 'Federated Search' page in the AiSOC interface. The left sidebar is identical to the previous screenshot, with 'Federated Search' selected. The main content area is titled 'Federated Search' with the subtitle 'One query, fanned out in parallel to every SIEM this tenant has connected, using that tenant's own stored credentials. Each backend answers independently.' Below this is a message: 'No SIEM connectors are connected. Federated search queries Splunk, Microsoft Sentinel, Elastic and QRadar connectors. Connect and enable at least one to search across them.' A 'Connect a SIEM' link is provided. The footer shows 'v10.0.0 - MIT License'.

Both say plainly that there is nothing, and the second names the four SIEMs it would query. Production never falls back to synthetic data: an unreachable backend makes the console name the failure rather than invent an investigation.

6. The REST API

518 operations across 417 paths. The specification is `docs/openapi.yaml` in the repository, regenerated on every change and gated against breaking changes in CI.

Authenticating

```
TOKEN=$(curl -s -X POST http://localhost:8000/api/v1/auth/login \
-H 'Content-Type: application/json' \
-d '{"email":"admin@aisoc.internal","password":"'$PASSWORD'"}' \
| python3 -c 'import json,sys; print(json.load(sys.stdin)["access_token"])')
```

The endpoints you will use first

Purpose	Call
List alerts	GET /api/v1/alerts
One alert with its investigation envelope	GET /api/v1/alerts/{id}
Create a case	POST /api/v1/cases
Launch an investigation	POST /api/v1/cases/{id}/investigate
Poll an investigation	GET /api/v1/cases/{id}/investigations/{run_id}
Query the lake	POST /api/v1/lake/sql
Propose a detection	POST /api/v1/detection-proposals
Configure SSO	POST /api/v1/sso-connections
Reopen a closed case	POST /api/v1/cases/{id}/reopen
Where the local model is running	GET /api/v1/llm/runtime
Test the tenant's provider credential	POST /api/v1/llm/credentials/test
Dashboard tiles for a window	GET /api/v1/metrics/dashboard?period=7d

Two contract details that cost people time:

- GET /api/v1/alerts returns the array under **items**, not **alerts**.
- On /metrics/dashboard, **alerts.total is open work, not everything ever received**. It counts **new**, **triaging** and **in_progress**, and the severity counts beside it are scoped the same way, because the console labels them Active Alerts and Critical — Require immediate action. Closed work is reported separately as **alerts.resolved**. Period-over-period **deltas** on /metrics/funnel are **percentages** — 5.0 means +5%, and a client that multiplies by 100 again renders -93.75 as -9375%.

Groups worth knowing about

Tag	Operations	What it covers
mssp	33	Child tenants, portfolio views, cross-tenant overrides
detection_rules	30	The rule catalogue, tuning, proposals, backtesting
cases	25	Cases, tasks, comments, evidence, investigations, reopen
scim	21	SCIM 2.0 user and group provisioning
graph	14	Entity graph traversal, blast radius, context import
connectors	13	Catalogue, instances, test-connection, scheduling
llm	6	Provider configuration, where the model runs, credential testing

Rate limits and errors

Errors carry a `detail` that names the cause. A refusal says what was refused and why — a 403 on an attribute condition tells you which condition, because a bare 403 is indistinguishable from a missing role and people debug the wrong thing for an hour.

7. MCP

AiSOC ships an MCP server, so an MCP-capable client can read alerts, cases and the decision ledger directly.

Connecting

```
{
  "mcpServers": {
    "aisoc": {
      "command": "python3",
      "args": ["-m", "app.server"],
      "cwd": "/path/to/AiSOC/services/mcp",
      "env": {
        "AISOC_API_URL": "http://localhost:8000",
        "AISOC_API_TOKEN": "your-token"
      }
    }
  }
}
```

The tools

Thirty-one, in four groups:

Group	Tools	For
Triage	<code>aisoc_list_alerts</code> , <code>aisoc_get_alert</code> , <code>aisoc_get_triage_verdict</code>	Reading the queue and what the agent concluded
Investigation	<code>aisoc_list_cases</code> , <code>aisoc_get_case</code> , <code>aisoc_run_investigation</code> , <code>aisoc_list_investigations</code>	Driving and following an investigation
Audit	<code>aisoc_replay_decision</code> , <code>aisoc_explain_step</code> , <code>aisoc_get_replay_report</code>	The prompt, the response and the tools used, for any step
Content and data	<code>aisoc_query_detections</code> , <code>aisoc_get_detection_rule</code> , <code>aisoc_lake_query</code> , <code>aisoc_lake_schema</code>	The rule catalogue and the event lake

Every agent decision is written to a persistent ledger, and `aisoc_replay_decision` and `aisoc_explain_step` are the auditable surface over it. That is the part worth using: you can ask the system to show you exactly what it was told and exactly what it said back.

A typical session

1. `aisoc_list_alerts` → find what is open.
2. `aisoc_get_alert` → read one, with its correlated siblings.
3. `aisoc_run_investigation` → have the agent work it.
4. `aisoc_explain_step` → audit any step it took.

8. Security posture

Control	State
Secrets	Generated per deployment, never committed
Connector credentials	Fernet-encrypted at rest; key rotation supported
Database access	Services connect as a DML-only role, so row-level security applies
Tenant isolation	Enforced at the query layer in every store, not by RLS alone
Route authorization	RBAC on every mutating route
Ingest	Authenticated; the tenant comes from the token
Prompts	Validated before they are sent. Raw logs, OCSF payloads and secret-shaped values are refused, not redacted afterwards
Outbound LLM	None by default. The bundled model runs beside the stack

A service with no credential refuses to serve rather than serving unauthenticated. That is the rule the posture rests on.

Enterprise SSO

SAML and OIDC, with the tenant and group mapping held on the connection an administrator configured rather than read from the assertion. An identity provider that can name its own tenant can name somebody else's.

```
curl -s -H "Authorization: Bearer $TOKEN" \
  http://localhost:8000/api/v1/sso-connections
```

If that returns `[]`, every SSO sign-in will 403 — there is no connection to resolve. Create one disabled, check the mapping, then enable it.

A mapping to `admin` or `platform_admin` is refused: those are reachable only through `bootstrap_admin`, and a group mapping would be a way back in.

9. Backup and disaster recovery

```
./scripts/backup.sh --component all
./scripts/restore.sh --timestamp 20260503T120000Z --component all
```

Six stores are covered. Two of them have deliberate behaviour worth knowing:

Redis asks you to confirm, and the default is not to restore it. It holds only state the platform rebuilds, and a restored cache serves stale answers with full confidence — worse than a cold start.

Neo4j replays Cypher rather than loading a dump, because `neo4j-admin load` needs the database stopped, and the platform degrades gracefully without the graph. Statements apply one at a time so a single failure does not discard the restore, and failures are counted and surfaced.

10. Honest limits

Stated plainly, because a guide that only lists strengths is a brochure.

- **No hosted LLM provider has been exercised.** Every published agent figure comes from a locally served model. The hosted path exists and is untested.
- **Long-duration scale figures do not exist.** The harness has ramp, burst, backpressure and 24/72-hour soak profiles; the long ones have not been run, and no throughput number is claimed from them.

- **Package distribution is blocked on credentials.** The release workflow builds and packs all eight npm and PyPI packages on every tag; the upload step is credential-gated. Install from source.
 - **Benchmark numbers are labelled.** Three of the four eval metrics are substrate self-consistency gates, not agent accuracy. Only MITRE accuracy measures the live agent, and the benchmark page says which is which.
 - **Response is not autonomous by default.** It requires explicit policy authorization and a human approver.
-

11. Where to go next

You want	Read
The full documentation portal	https://beenuar.github.io/AiSOC/
The API specification	docs/openapi.yaml
What changed and when	CHANGELOG.md
What is gated and by which job	docs/audit/CLAIM_TO_GATE_MATRIX.md
To report a vulnerability	SECURITY.md
To contribute	CONTRIBUTING.md

Credits

Development of AiSOC is **funded and supported by Cyble** (<https://cyble.com>), who pay for the engineering time behind the project and release it under the MIT licence rather than keeping it.

That funding buys no special treatment in the product: there are no Cyble-only features, no gated modules, and no telemetry reporting back to anyone. The same code, under the same licence, for everybody.